

云原生训练营

模块十三： Kubernetes 集群联邦和 Istio 多集群管理

孟凡杰

前 eBay 资深架构师



目录

1. 多集群治理的驱动力
2. 集群联邦
3. Clusternet
4. Istio 多集群

分布式云是未来

- 成本优化 (Cost Effective)
- 更好的弹性及灵活性 (Elasticity & Flexibility)
- 避免厂商锁定 (Avoid Vendor Lock-in)
- 第一时间获取云上的新功能 (Innovation)
- 容灾 (Resilience & Recovery)
- 数据保护及风险管理 (Data Protection & Risk Management)
- 提升响应速度 (Network Performance Improvements)

分布式云的挑战

- **Kubernetes** 单集群承载能力有限
- 异构的基础设施
- 存量资源接入
- 配置变更及下发
- 跨地域、跨机房应用部署及管理
- 容灾与隔离性，异地多活
- 弹性调度及自动伸缩
- 监控告警

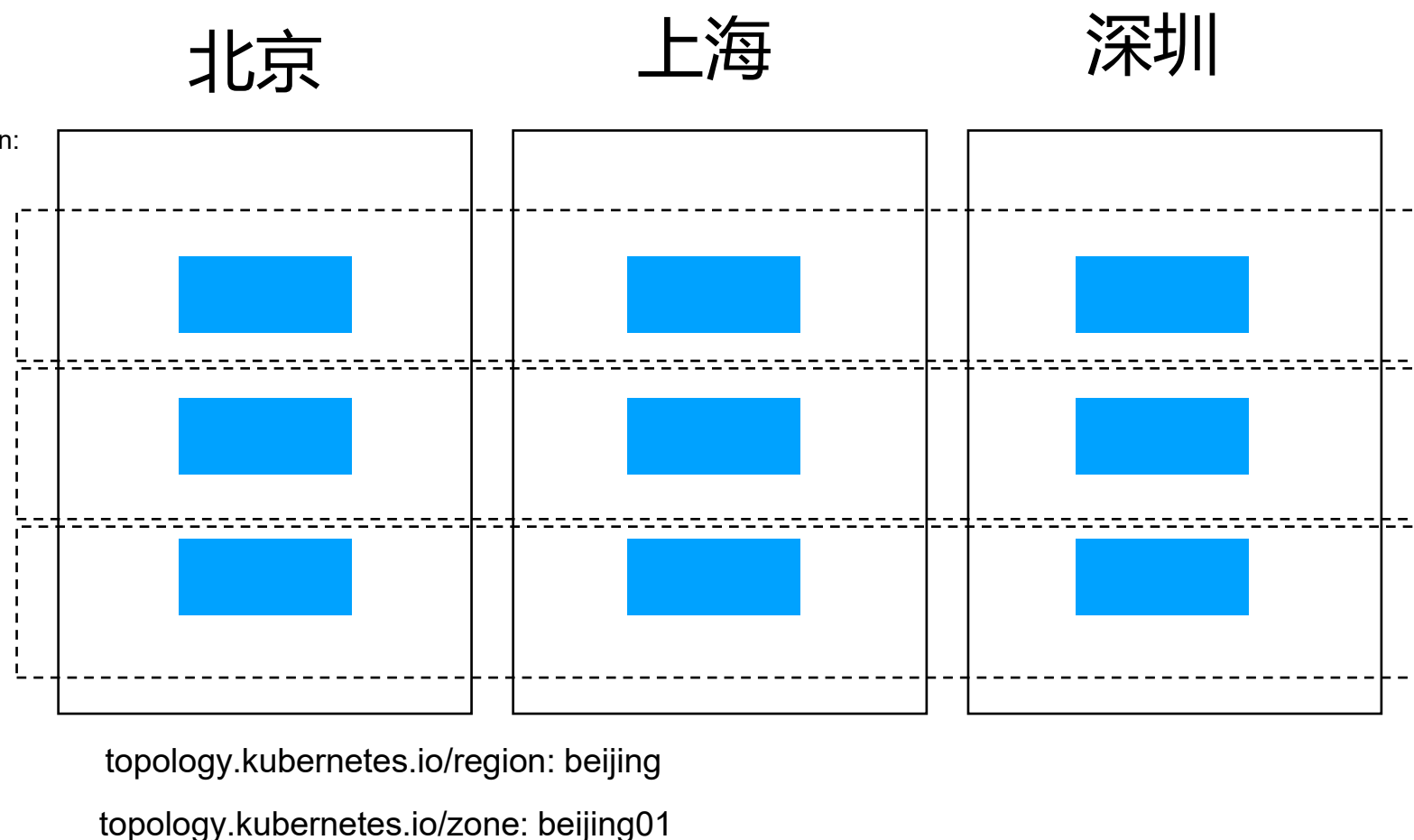
如何应对

- 通过 Kubernetes 屏蔽底层基础设施，提供统一的接入层
- 多云架构
 - 多集群 $\neq$ 多云
 - 多集群管控
- 统一的管控面
- 方便接入，降低使用门槛

跨地域的集群管理

```
apiVersion: v1
kind: Pod
metadata:
  name: with-pod-affinity
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: region
                operator: In
                values:
                  - beijing
          topologyKey: topology.kubernetes.io/zone
```

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
  topologyKeys:
    - "topology.kubernetes.io/zone"
    - "topology.kubernetes.io/region"
```



1. 集群联邦

集群联邦的必要性

- 单一集群的管理规模有上限

- 数据库存储

- etcd 作为 Kubernetes 集群的后端存储数据库，对空间大小的要求比较苛刻，这限制了集群能存储的对象数量和大小。

- 内存占用

- 为提高系统效率，Kubernetes 的 API Server 作为 API 网关，会对该集群的所有对象做缓存。集群越大，缓存需要的内存空间就越大。
 - 其他 Kubernetes 控制器也需要对侦听的对象构建客户端缓存，这些都需要占用系统内存。这些内存需求都要求对系统的规模有所限制。

- 控制器复杂度

- Kubernetes 的一个业务流程是由多个对象和控制器联动完成的，即使控制器遵循了设计原则，随着对象数量的增长，控制器的处理耗时也会越来越长。

集群联邦的必要性

- 单个计算节点资源上限

- 单个计算节点的资源，不仅仅是 CPU、内存等可量化资源。
- 还有端口、进程数量等不可量化资源。比如 Linux 支持的 TCP 端口上限是 65535，去除常用端口和程序源端口后，留给 Service nodePort 的端口数量是有限的，这限制了集群支持的 Service 的数量。

- 故障域控制

- 集群规模越大，控制平面组件出现故障时的影响范围就越大。为了更好地控制故障域（Fault Domain），需要将大规模的数据中心切分成多个规模相对较小的集群，每个集群控制在一定规模。

- 应用高可用部署

- 生产应用通常需要多数据中心部署来保障跨地域高可用，以确保当其中一个数据中心出现故障，或者集群做技术迭代更新时，其他数据中心可以继续提供服务。

- 混合云

- 私有云加公有云的混合云模式逐渐成为企业的主流架构。

集群联邦的职责

- 跨集群同步资源

- 联邦可以将资源同步到多个集群并协调资源的分配。例如，联邦可以保证一个应用的 Deployment 被部署到多个集群中，同时能够满足全局的调度策略。

- 跨集群服务发现

- 联邦汇总各个集群的服务和 Ingress，并暴露到全局 DNS 服务中。

- 高可用

- 联邦可以动态地调整每个集群的应用实例，且隐藏了具体的集群信息。

- 避免厂商锁定

- 每个集群都是部署在真实的硬件或云供应商提供的硬件（或虚拟硬件）之上的，若要更换供应商，只需在新供应商提供的硬件上部署新的集群，并加入联邦。联邦可以几乎透明地将应用从原集群迁移到新集群而无须对应用做更改。

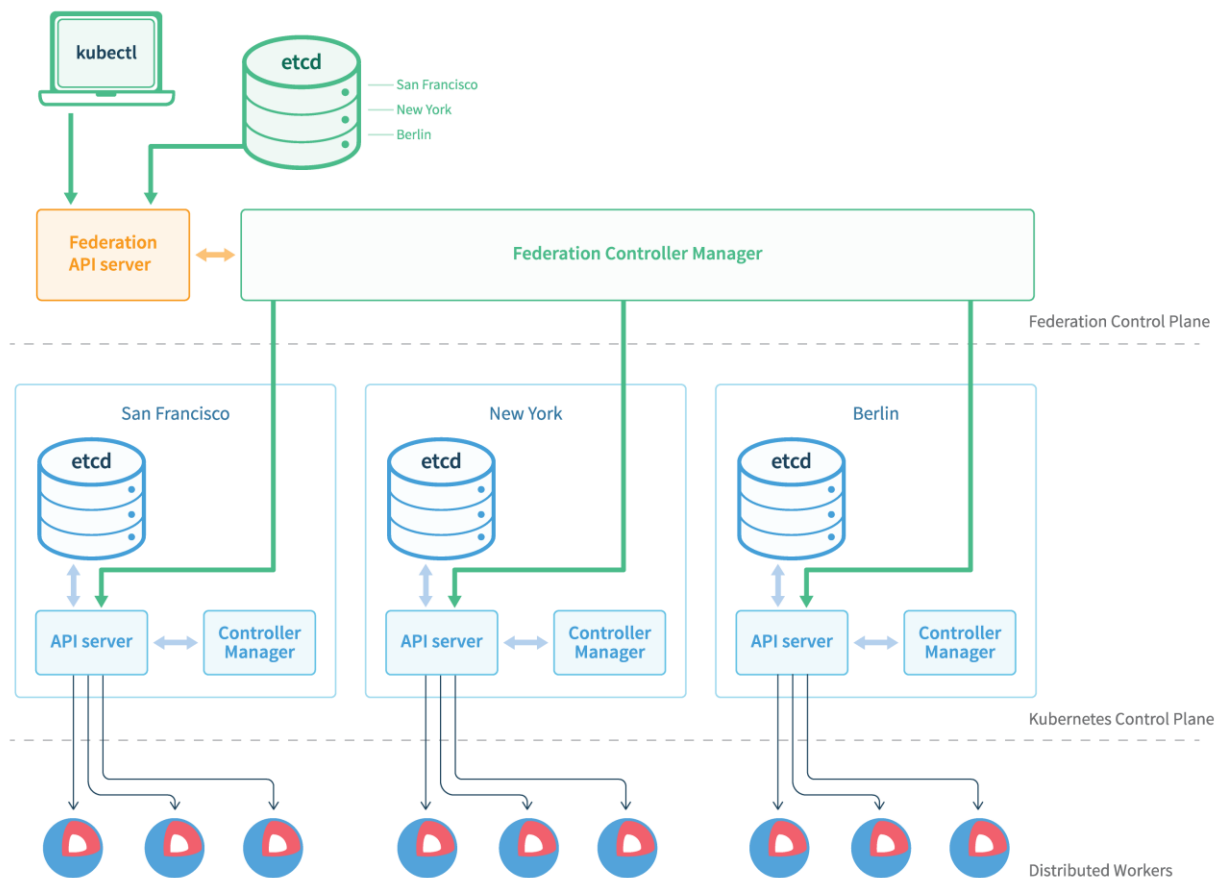
混合云

- 混合云是指将公有云和私有云整合在一起的统一云平台。
- 用户业务可灵活部署或扩容在不同云中。
- 混合云避免厂商锁定。
- 混合云可支持更多复杂业务场景：
 - Cloud Burst。
 - 如 12306，低流量时运行在本地数据中心，当请求量突然爆发的时候，可以直接在公有云扩容，节省数据中心成本。
 - 可将业务分为包含敏感数据和不包含敏感数据的业务，将敏感数据业务运行在私有云，将非敏感数据业务运行在公有云。
 - 可重复利用公有云提供商数据中心靠近边缘的能力。

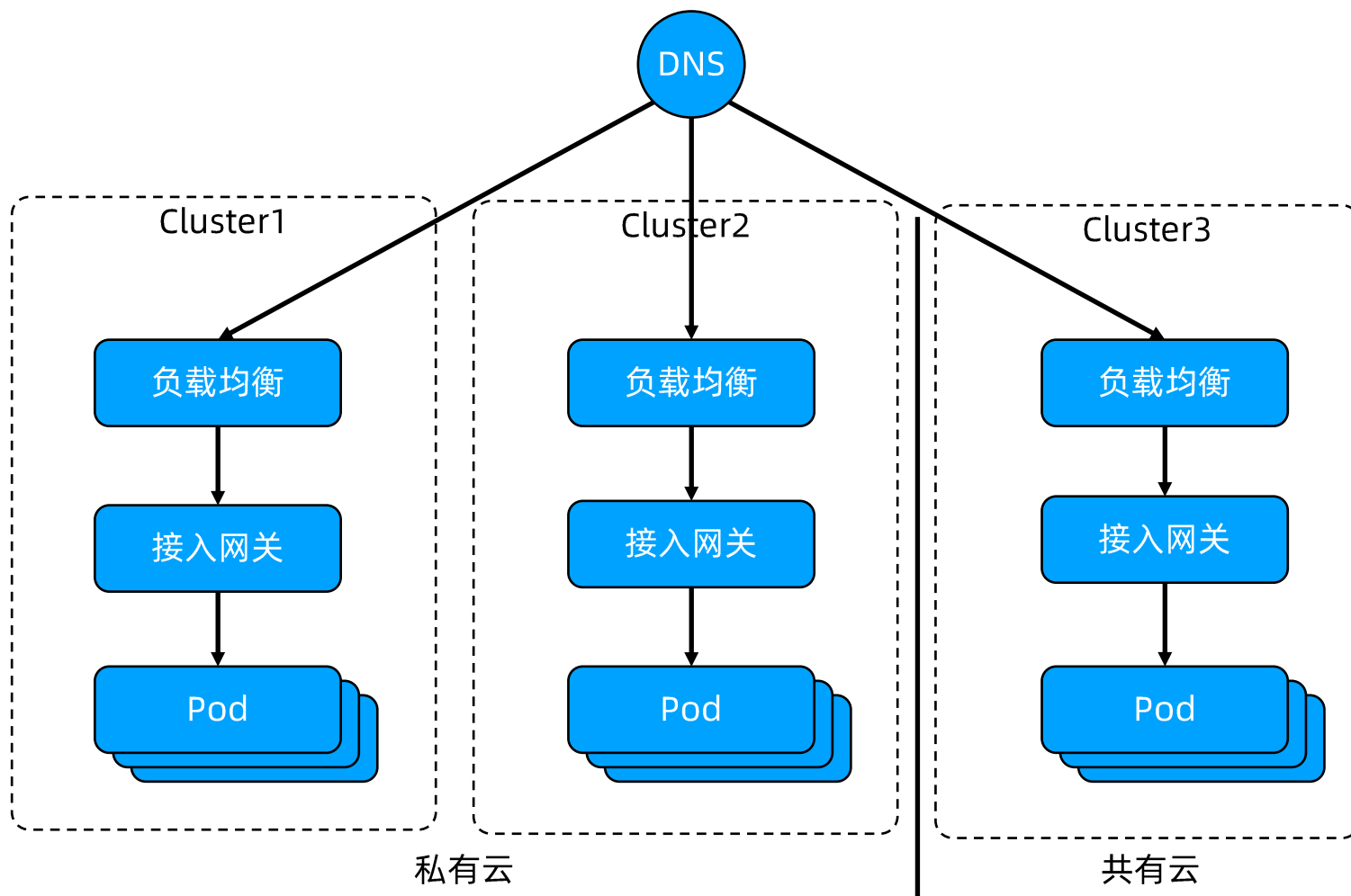
集群联邦

集群联邦（Federation）是将多个 Kubernetes 集群注册到统一控制平面，为用户提供统一 API 入口的多集群解决方案。

集群联邦设计的核心是提供在全局层面对应用的描述能力，并将联邦对象实例化为 Kubernetes 对象，分发到联邦下辖的各个成员集群中。



基于集群联邦的高可用应用部署



集群联邦核心架构

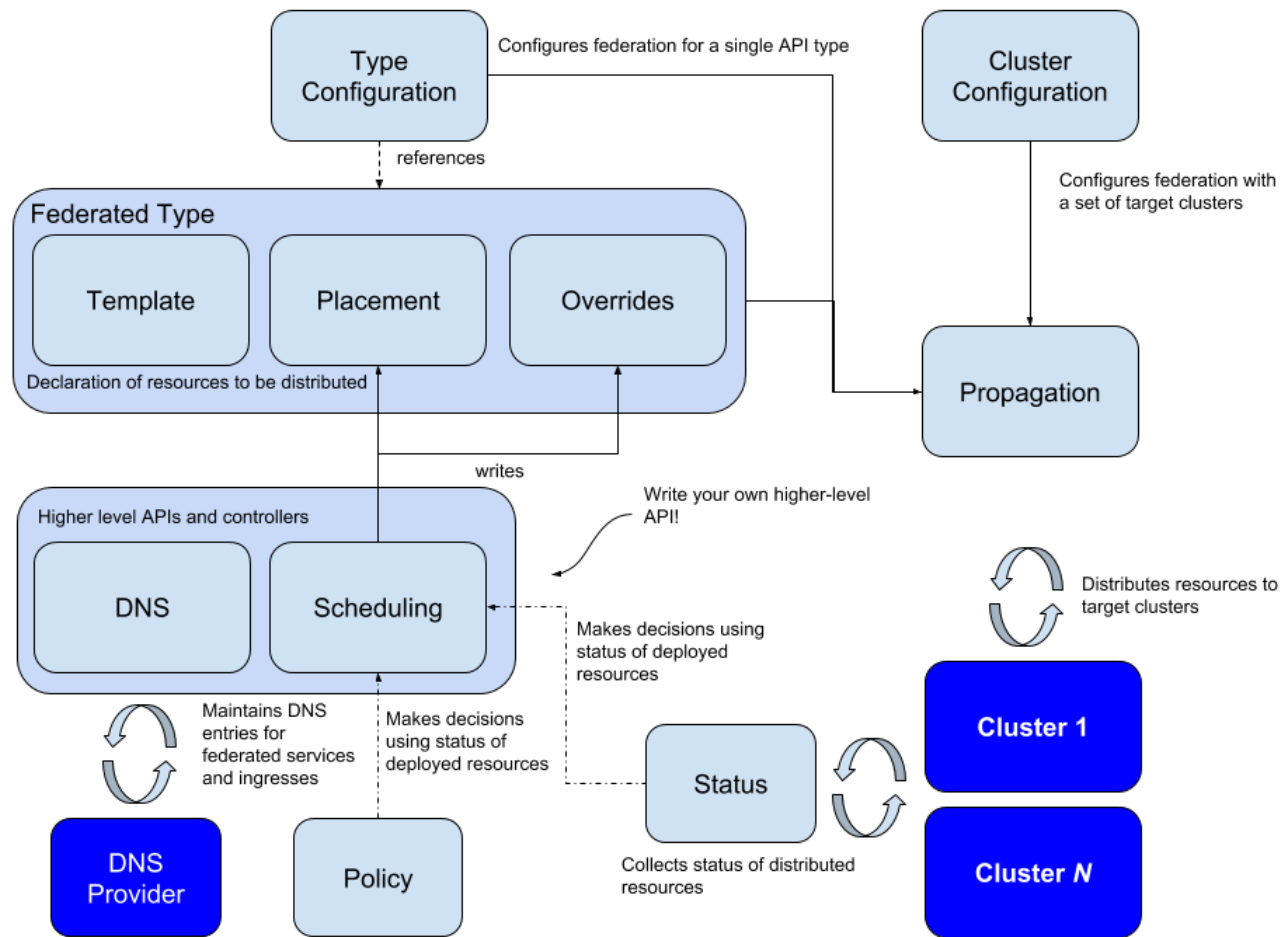
etcd 作为分布式存储后端存储所有对象；

API Server 作为 API 网关，接收所有来自用户及控制平面组件的请求；

不同的控制器对联邦层面的对象进行管理、协调等；

调度控制器在联邦层面对应用进行调度、分配。

集群联邦支持灵活的对象扩展，允许将基本 Kubernetes 对象扩展为集群联邦对象，并通过统一的联邦控制器推送和收集状态。



集群联邦管理的对象

成员集群是联邦的基本管理单元，所有待管理集群均须注册到集群联邦。

集群联邦 V2 提供了统一的工具集（Kubefedctl），允许用户对单个对象动态地创建联邦对象。

动态对象的生成基于 CRD。

	Kubernetes 集群	Kubernetes 集群联邦
计算资源	Node	Cluster
部署	Deployment	FederatedDeployment
配置	Configmap	FederatedConfigmap
密钥	Secret	FederatedSecret
任何其他 Kubernetes 对象	Foo	FederatedFoo

集群注册中心

- 集群注册中心（ClusterRegistry）提供了所有联邦下辖的集群清单，以及每个集群的认证信息、状态信息等。
- 集群联邦本身不提供算力，它只承担多集群的协调工作，所有被管理的集群都应注册到集群联邦中。
- 集群联邦使用了单独的 KubeFedCluster 对象（同样适用 CRD 来定义）来管理集群注册信息：
 - 在该对象的定义中，不仅包含集群的注册信息，还包含集群的认证信息的引用，以明确每个集群使用的认证信息；
 - 该对象还包含各个集群的健康状态、域名等；
 - 当控制器行为出现异常时，直接通过集群状态信息即可获知控制器异常的原因。

安装

- 下载 **federation** 代码;

git clone: <https://github.com/kubernetes-sigs/kubefed.git>

- 选择 HostCluster, 确认 kubeconfig 符合 federation 命名规范, 用 vi 编辑 kubeconfig, 确保 context 属性没用 @ 字符;

```
vi ~/.kube/config
contexts:
- context:
  cluster: kubernetes
  user: kubernetes-admin
  name: cluster1
current-context: cluster1
```

- 安装;

```
cd kubefed/
./scripts/deploy-kubefed-latest.sh
```

- 安装完成后, 相关资源均可在 **kube-federation-system** 查看。

集群类型

Host:

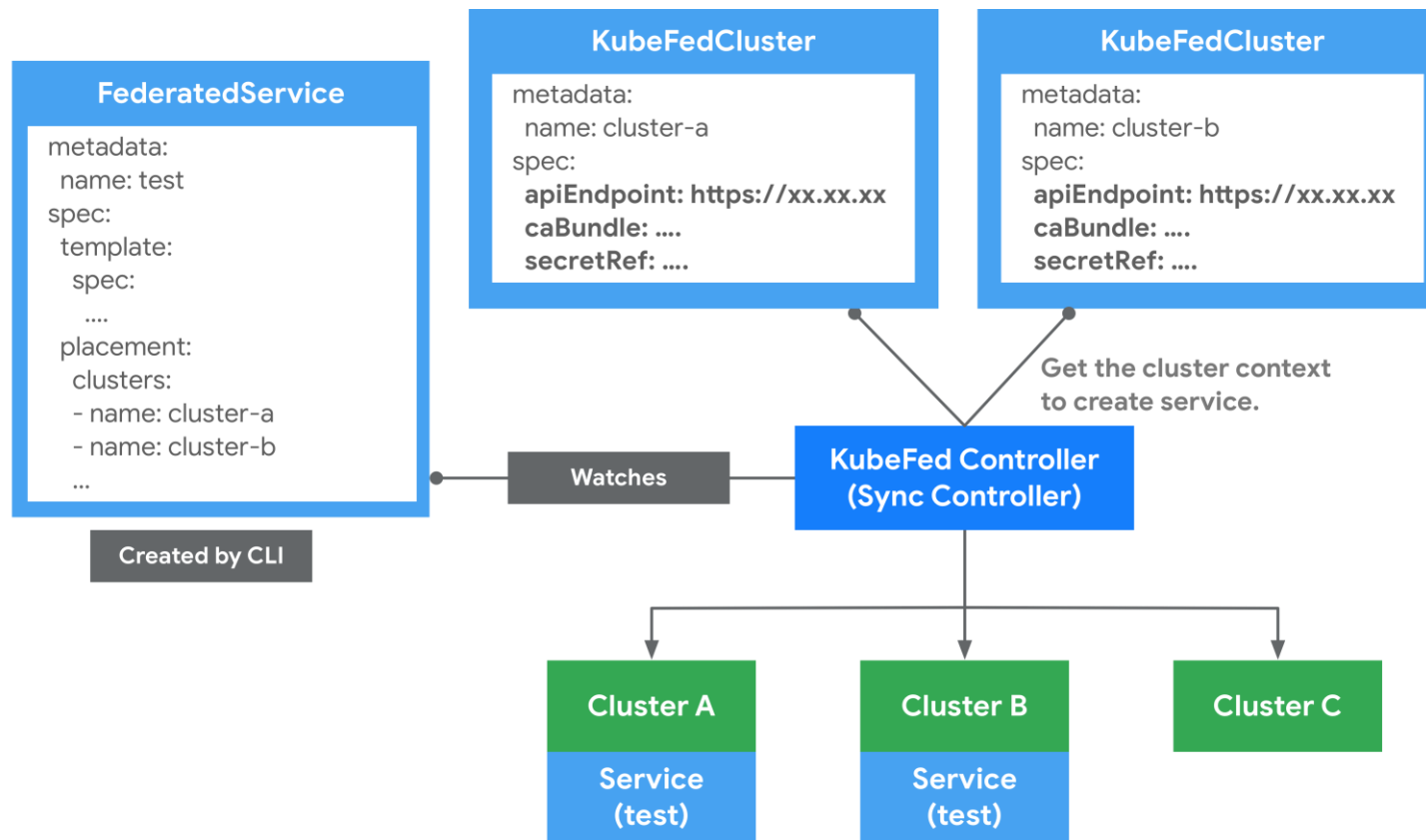
用于提供 KubeFed API 与控制平面的集群，本质上就是 Federation 控制面。

Member:

通过 KubeFed API 注册的集群，并提供相关身份凭证来让 KubeFed Controller 能够存取集群。
Host 集群也可以作为 Member 被加入。

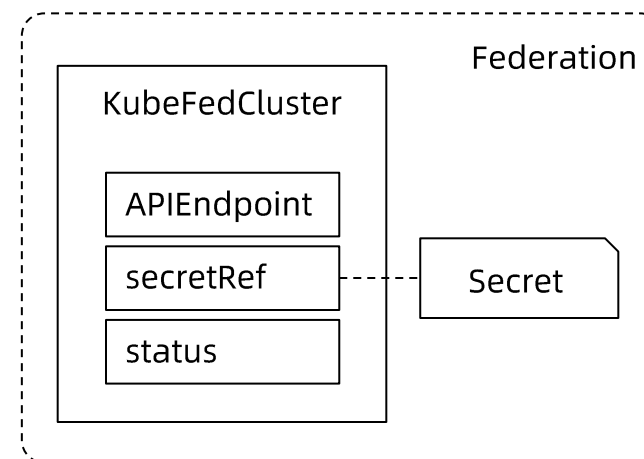
注册集群 KubeFedCluster

- 用来定义哪些 Kubernetes 集群要被联邦。
- 可透过 **kubefedctl join/unjoin** 来加入/删除集群，当成功加入时，会建立一个 KubeFedCluster 组件来储存集群相关信息，如 **API Endpoint**、**CA Bundle** 等。
- 这些信息会被用在 **KubeFed Controller** 存取不同 Kubernetes 集群上，以确保能够建立 Kubernetes API 资源。



KubeFedCluster

```
apiVersion: core.kubefed.io/v1beta1
kind: KubeFedCluster
metadata:
  name: "cluster1"
spec:
  apiEndpoint: https://API Server.cluster1.example.com
  caBundle: LS0tLS****LS0K
  disabledTLSValidations:
    - '*'
  secretRef:
    name: cluster1-vhvw
status:
  conditions:
    - reason: ClusterReady
    type: Ready
  region: China
  zones:
    - Shanghai
```



Federation 支持的核心对象

API Group	用途
core.kubefed.k8s.io	集群组态、联邦资源组态、KubeFed Controller 设定档等。
types.kubefed.k8s.io	被联邦的 Kubernetes API 资源。
scheduling.kubefed.k8s.io	副本编排策略。
multiclusterdns.kubefed.k8s.io	跨集群服务发现设定。

Type Configuration

- 定义了哪些 Kubernetes API 资源要被用于联邦管理。
- 若想新增 Federated API 的话，可通过 `kubefedctl enable <res>` 指令来建立。
- 比如说想将 ConfigMap 资源通过联邦机制建立在不同集群上时，就必须先在 **Federation Host** 集群中，通过 CRD 建立新资源 **FederatedConfigMap**，接着再建立名称为 configmaps 的 Type configuration (**FederatedTypeConfig**) 资源，然后描述 ConfigMap 要被 FederatedConfigMap 所管理，
- 这样 KubeFed Controllers 才能知道如何建立 Federated 资源。

FederatedConfigMap

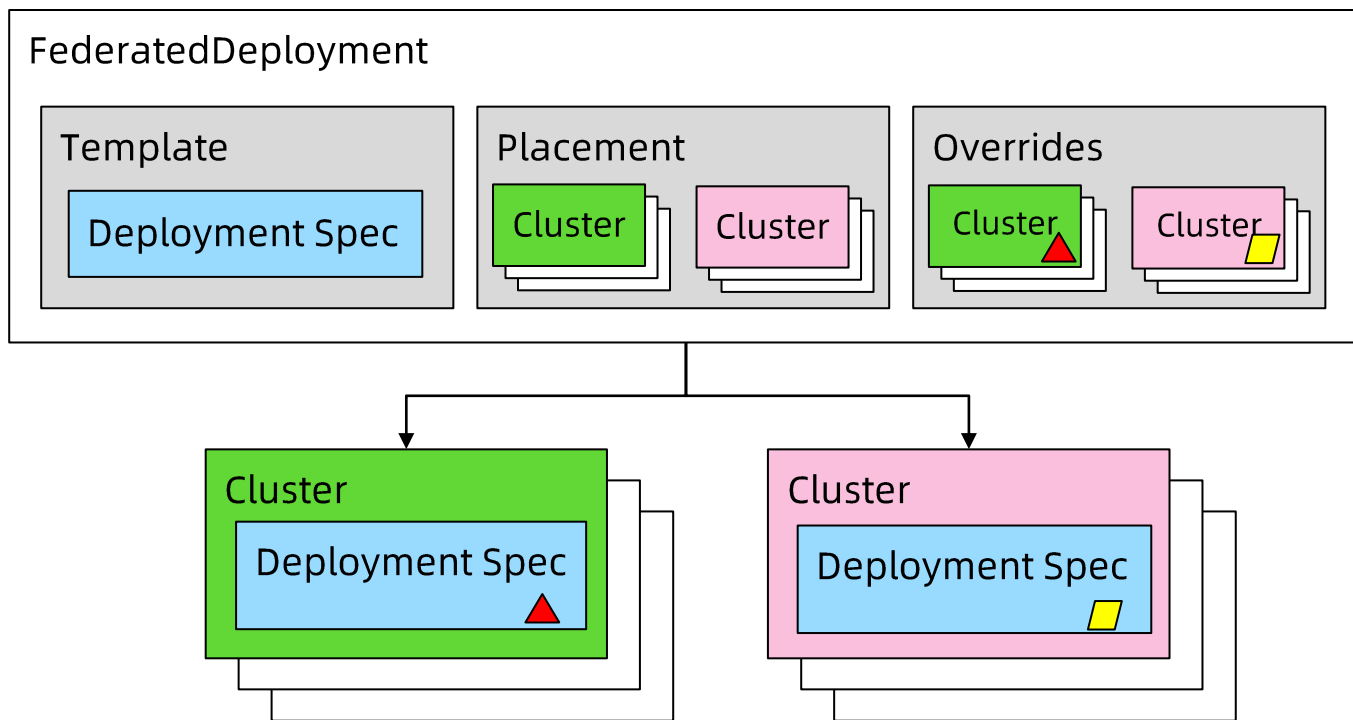
```
k get crd federatedconfigmaps.types.kubefed.io
```

NAME	CREATED AT
federatedconfigmaps.types.kubefed.io	2021-09-13T10:10:03Z

```
apiVersion: core.kubefed.io/v1beta1
kind: FederatedTypeConfig
metadata:
  name: configmaps
spec:
  federatedType:
    group: types.kubefed.io
    kind: FederatedConfigMap
    pluralName: federatedconfigmaps
    scope: Namespaced
    version: v1beta1
  propagation: Enabled
  targetType:
    kind: ConfigMap
    pluralName: configmaps
    scope: Namespaced
    version: v1
```

联邦对象组成

- Template: 定义 Kubernetes 对象的模板。
- Placement: 定义联邦对象需要被同步的目标集群。
- Overrides: 不同目标集群中，对 Kubernetes 对象模板的本地化属性。



Template

Template 是联邦对象中定义 Kubernetes 集群对象的模板部分，它的内容为完整的 Kubernetes 对象。

```
apiVersion: types.federation.k8s.io/v1alpha1
kind: FederatedDeployment
metadata:
  name: sample-federated-deployment
spec:
  template:
    metadata:
    spec:
      replicas: 2
      template:
        spec:
          containers:
            - image: nginx
              name: nginx
placement:
# ...
overrides:
# ...
```

Placement

- **Placement** 用来配置联邦对象的目标集群，其值可以是具体的集群名单，也可以是 clusterSelector 选择对应标签（label）的集群。
- 当两者同时存在时，明确定义的集群名单具有较高优先级。
- 联邦根据优先级来定义要同步对象的目标集群，如果提供了集群名单（哪怕是一个空 List），则无论 clusterSelector 提供什么内容，都会被忽略。

```
apiVersion: types.federation.k8s.io/v1alpha1
kind: FederatedDeployment
metadata:
  name: sample
  namespace: sample
spec:
  template:
    # .....
  placement:
    clusters:
      - name: cluster1
      - name: cluster2
    clusterSelector:
      matchLabels:
        region: china
        zone: shanghai
  overrides:
    #....
```

Overrides

- **Overrides** 用于针对每个集群进行本地化定制。
- 在实际部署应用时，通常会通过调整不同集群中的配置模板，部署符合特定集群需求的应用，以更好地发挥网络、计算资源、存储等的优势。
- 目前 Overrides 不支持 List(Array)。比如说无法修改
`spec.template.spec.containers[0].image`。

思考一下为什么？

```
apiVersion: types.federation.k8s.io/v1alpha1
kind: FederatedDeployment
metadata:
spec:
  template:
    spec:
      replicas: 2
      template:
        # ....
  placement:
    clusters:
      - name: cluster1
      - name: cluster2
  overrides:
    - clusterName: cluster2
      clusterOverrides:
        - path: /spec/replicas
          value: 3
```

使用 Federated 对象

将 Namespace 设置为联邦对象

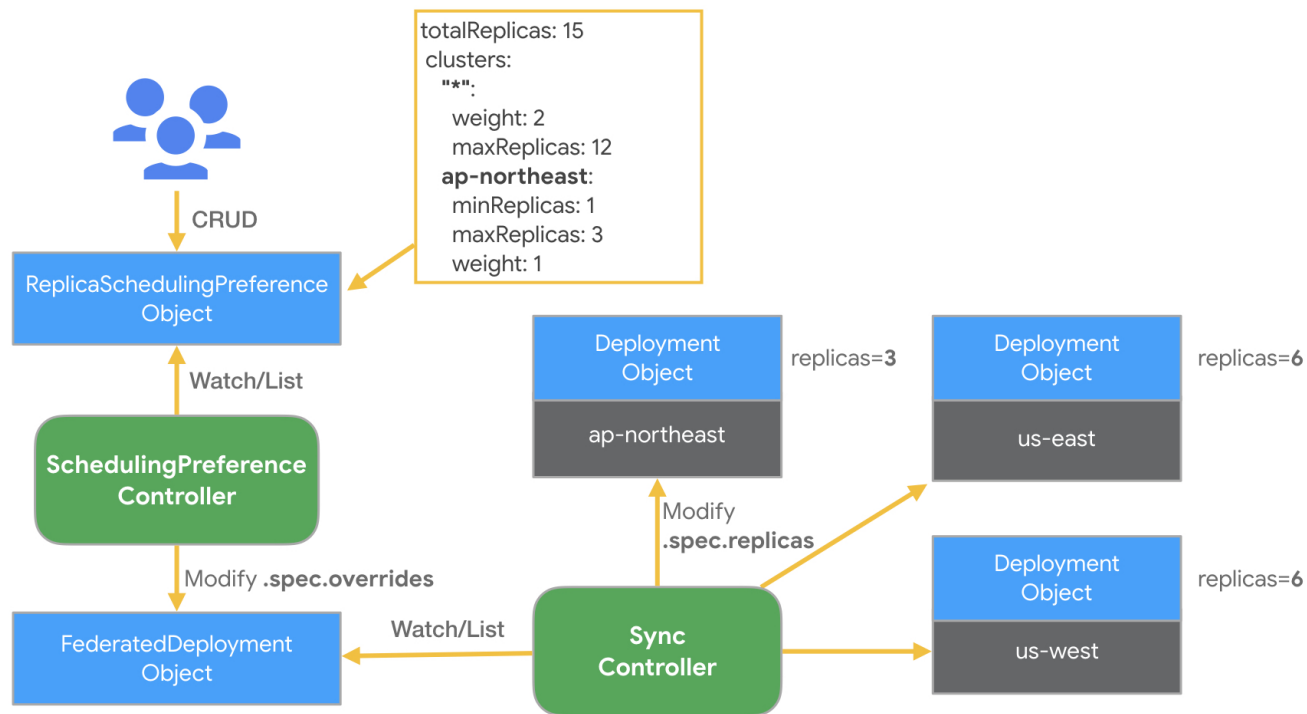
```
kubefedctl federate ns default
```

联邦调度

KubeFed 提供了一种自动化机制来将工作负载实例分散到不同的集群中，这能够基于总副本数与集群的定义策略来将 Deployment 或 ReplicaSet 资源进行编排。

编排策略是通过建立

ReplicaSchedulingPreference (RSP) 文件，再由 KubeFed RSP Controller 监听与撷取 RSP 内容来将工作负载实例建立到指定的集群上。

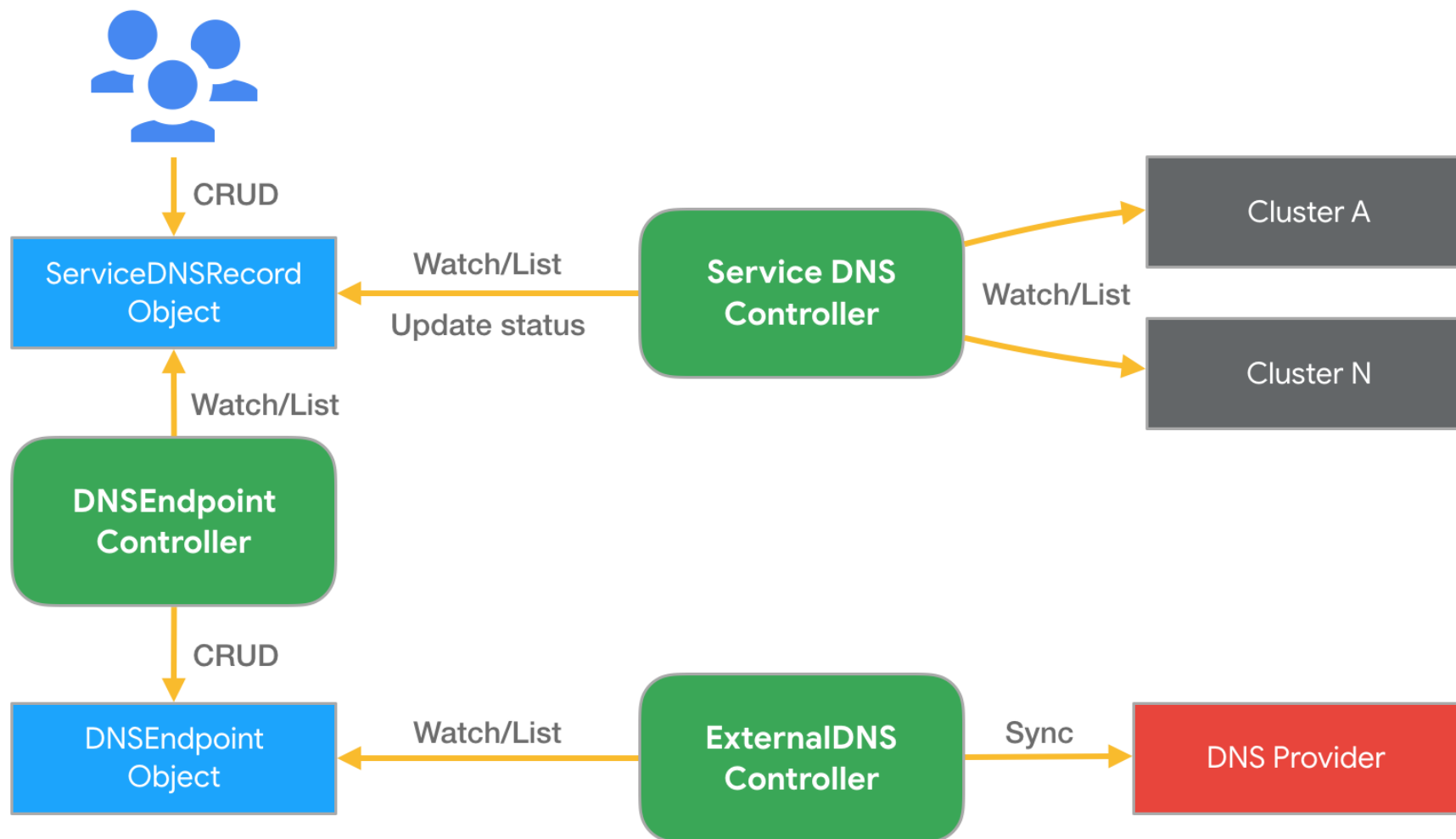


多集群 DNS

- KubeFed 提供了一组 API 资源，以及 Controllers 来实现跨集群 Service/Ingress 的 DNS records 自动产生机制。
- 需要结合 ExternalDNS 来同步更新至 DNS 服务供应商。
- 在 2020 年被移出，这里提供一个 DNS 接入的思路。

Kubefed v2 should be only focused on the federation of resources across kubefed clusters. In the Kubernetes ecosystem there are other third party tools (such as service meshes) that already provide support to the DNS based federated ingress.

ServiceDNSRecord 原理



2. Clusternet

Clusternet 简介

Clusternet (Cluster Internet) 是一个兼具多集群管理和跨集群应用编排的开源云原生管控平台，打通了跨 VPC、跨地域、跨云的集群管理。

Clusternet 面向未来混合云、分布式云和边缘计算场景设计，支持海量集群的接入和管理。

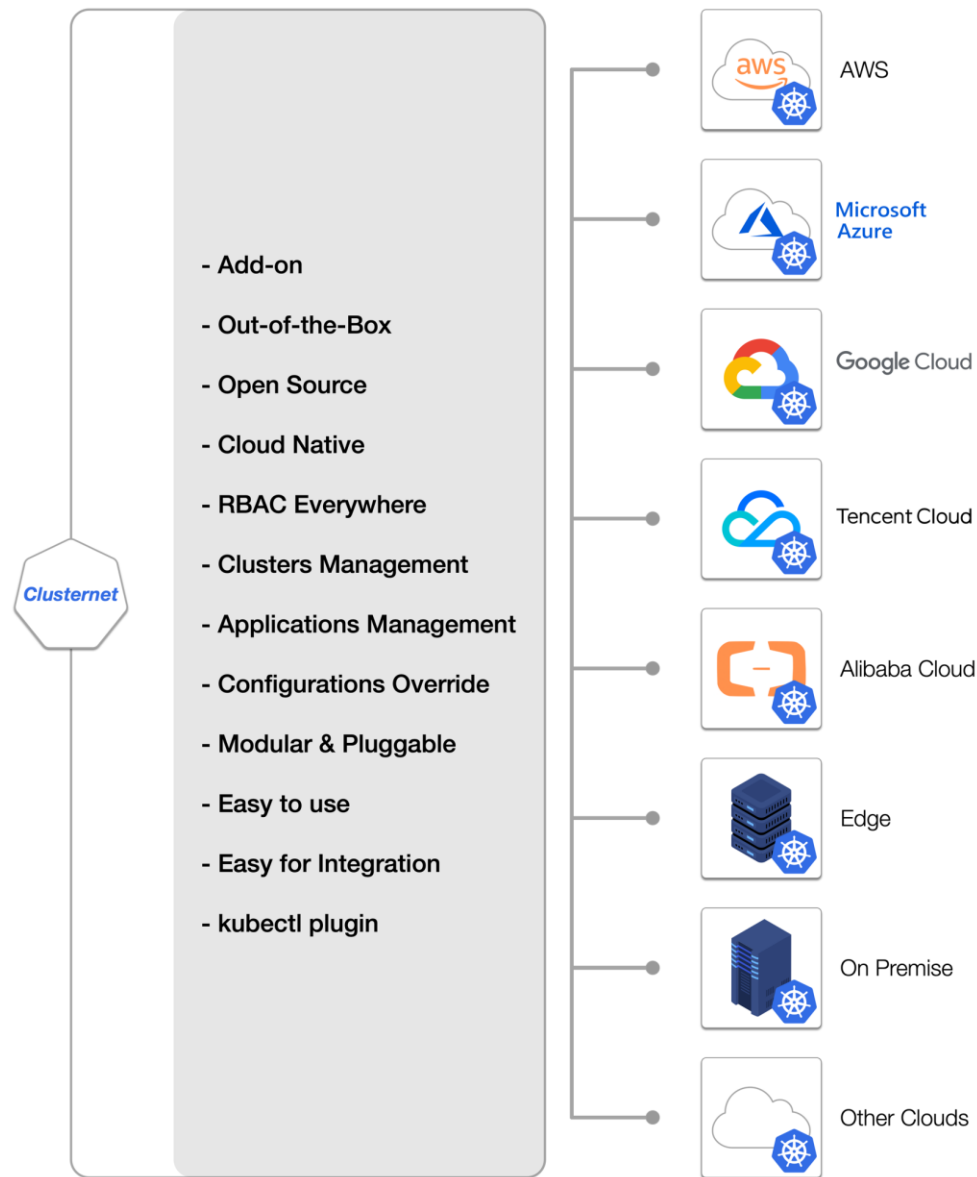
Clusternet 在保证无侵入且轻量化的基础上，创建了一张集群网络，对子集群进行纳管，并支持多集群的应用编排与治理。

开源项目地址：<https://github.com/clusternet/clusternet>

- 最新版本 v0.6.0
- 多平台支持：linux/amd64, linux/arm64, linux/ppc64le, linux/s390x, linux/386, linux/arm

Clusternet 能力一览

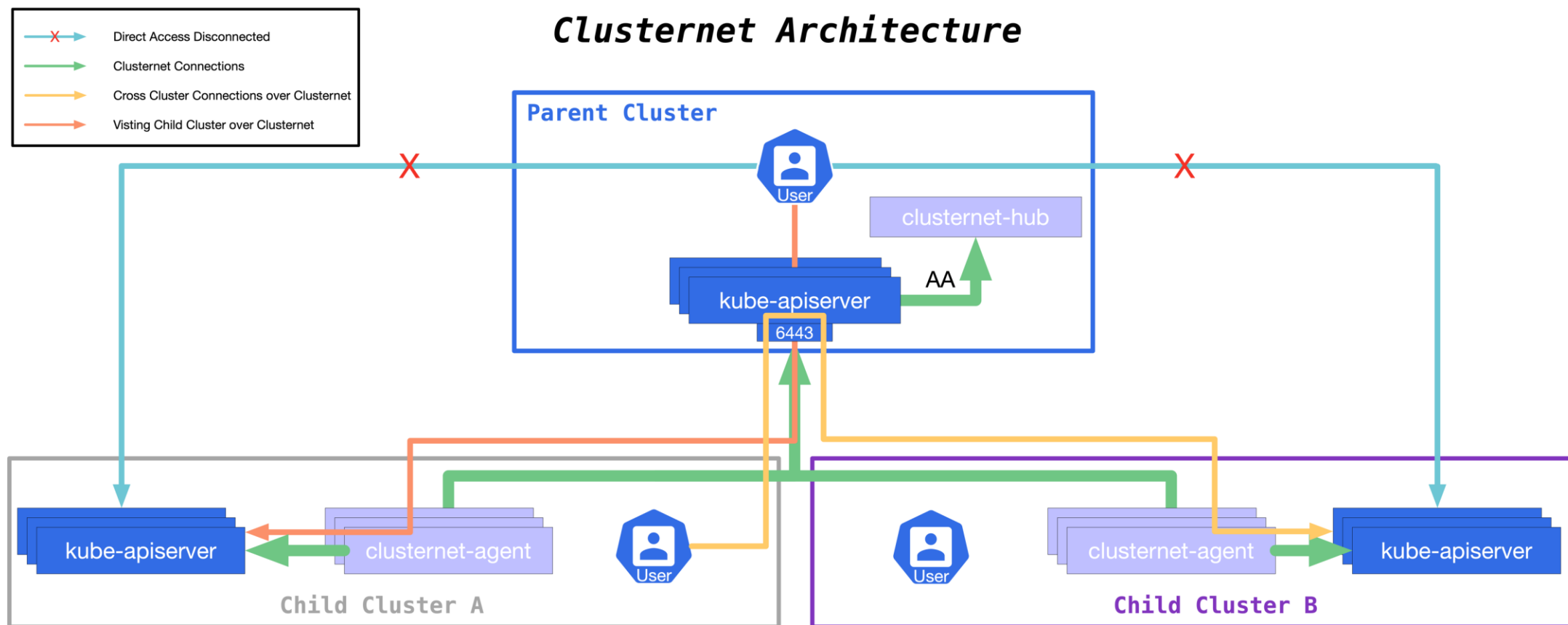
- 统一管控各类 **Kubernetes** 集群；
- 集群管理 **Pull / Push** 模式；
- 轻量化，开箱即用，易于部署和维护；
- 跨集群的服务发现及服务互访；
- Kubernetes 原生，没有额外的学习成本；
- 完善的 **RBAC** 能力，访问任一子集群；
- 完善的接入能力：**kubectl plugin / client-go**；
- 支持分发各类原生应用 / **CRD / HelmChart**。



多集群管理的挑战

- 集群无处不在
- 公有云、私有云、混合云、边缘、裸金属
- 提供一致的纳管能力
 - 集群一键注册能力
 - 一致性的集群访问体验, 比如 exec, logs
 - 权限管控 RBAC
- 架构要足够轻量化, 方便一键接入

Clusternet 架构



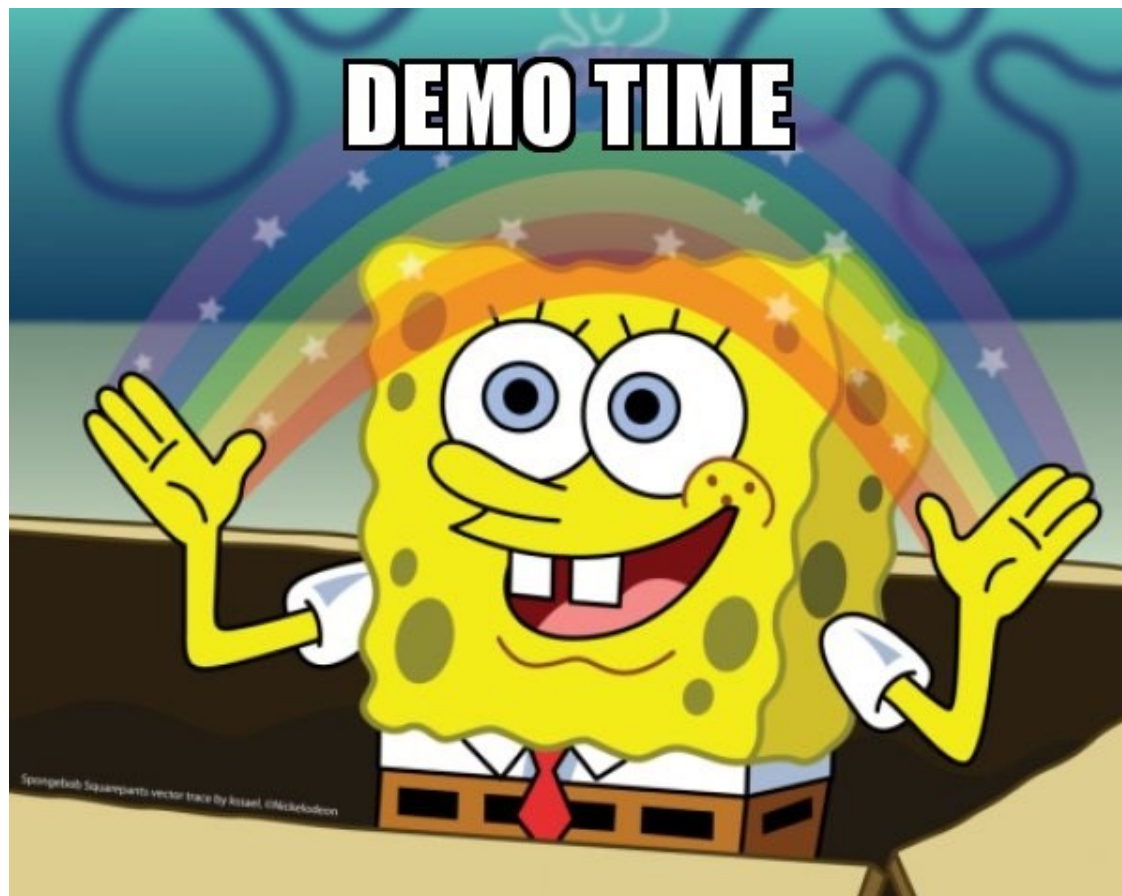
Clusternet 架构

- 最轻量化的架构
- 一站式连接各类集群
- 支持 Pull 和 Push 模式
- 跨集群路由访问
- 支持子集群 RBAC 访问
- 提供一致的集群访问体验，比如 exec, logs
- 原生 Kubernetes API
- 接入成本低，kubectl plugin / client-go

如何通过 Clusternet 访问任一子集群

- Clusternet 支持通过 Bootstrap Token, Service Token, TLS 证书等 RBAC 的方式访问子集群;
 - 子集群的 credential 信息不需要存在父集群中
- 也支持通过 kubectl 命令行的方式对子集群进行 create/get/list/watch/patch/exec 等操作。

如何通过 Clusternet 访问任一子集群



如何通过 Clusternet 访问任一子集群

更详细步骤及功能可以参照

<https://github.com/clusternet/clusternet/blob/main/docs/tutorials/visiting-child-clusters-with-rbac.md>

- 使用 curl 进行访问
- 通过 kubectl 进行访问

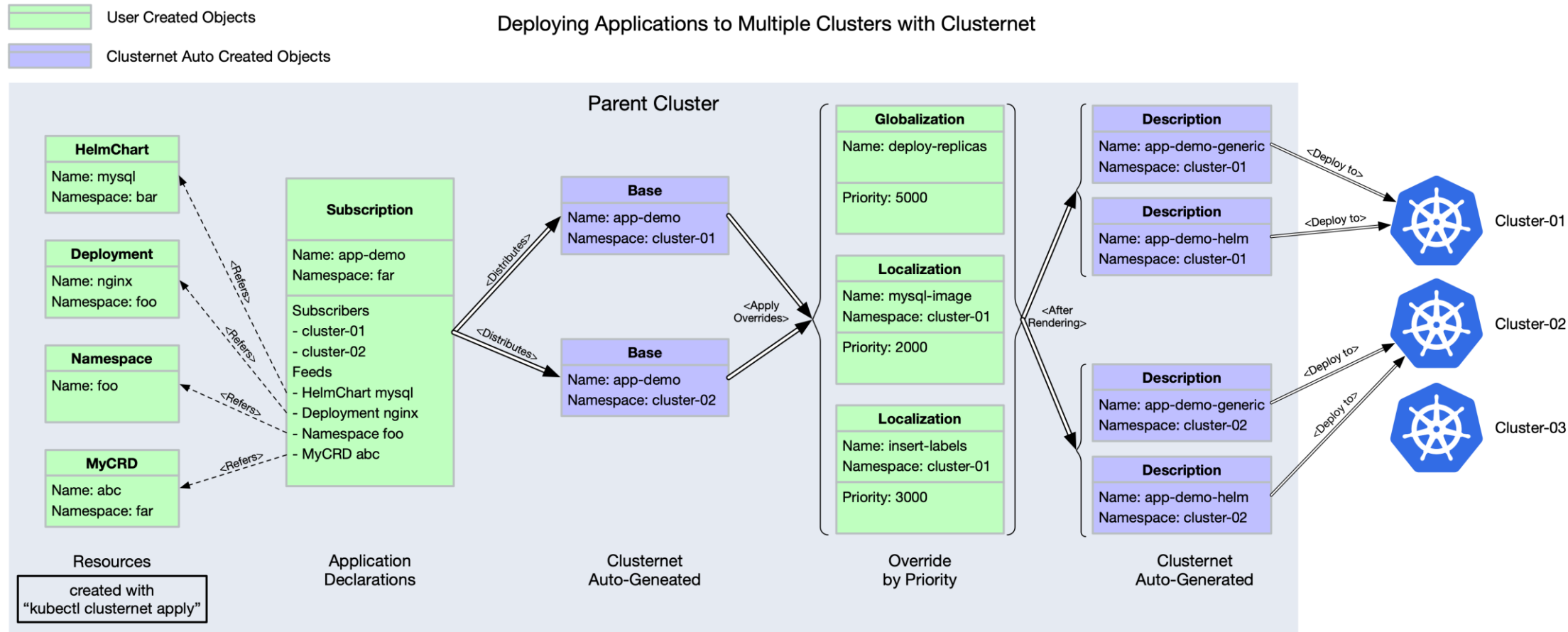
Clusternet 应用分发设计

- 完全兼容 Kubernetes 的内置 API
- 支持 Helm Charts
- 支持 CRD
- 丰富灵活的分发策略配置、差异化策略

应用分发的差异化痛点

1. 在分发的资源上全部打上统一的标签，比如 `apps.my.company/deployed-by: my-platform`;
2. 在分发到子集群的资源上标记集群的信息，比如 `apps.my.company/running-in: cluster-01`;
3. 调整应用在每个集群中的副本数目、镜像名称等;
4. 在分发到某集群前，调整应用在该集群中的一些配置，比如注入一个 `Sidecar` 容器等;
5. 灰度升级，变更可控，方便回滚;
6. 重复定义怎么办？冲突吗？
7. ...

Clusternet 应用分发模型



如何创建一个资源

1. kubectl 插件 kubectl-clusternet

```
$ kubectl krew update  
$ kubectl krew install clusternet  
# check plugin version  
$ kubectl clusternet version
```

- Live demo

2. 通过 client-go wrapper

- 一键接入，无需改造
- 兼容 client-go 各个版本
- 示例：

<https://github.com/clusternet/clusternet/blob/main/examples/clientgo/README.md>

示例：声明多集群应用 Subscription

```
apiVersion: apps.clusternet.io/v1alpha1
kind: Subscription
metadata:
  name: app-demo
  namespace: default
spec:
  subscribers: # defines the clusters to be distributed to
    - clusterAffinity:
        matchLabels:
          clusters.clusternet.io/cluster-id: dc91021d-2361-4f6d-a404-7c33b9e01118
  feeds: # defines all the resources to be deployed with
    - apiVersion: apps.clusternet.io/v1alpha1
      kind: HelmChart
      name: mysql
      namespace: default
    - apiVersion: v1
      kind: Namespace
      name: foo
    - apiVersion: apps/v1
      kind: Service
      name: my-nginx-svc
      namespace: foo
    - apiVersion: apps/v1
      kind: Deployment
      name: my-nginx
      namespace: foo
```

示例：差异化策略 Localization/Globalization

```
apiVersion: apps.clusternet.io/v1alpha1
kind: Localization
metadata:
  name: nginx-local-overrides-demo-lower-priority
  namespace: clusternet-5l82l # PLEASE UPDATE THIS TO YOUR ManagedCluster NAMESPACE!!!
spec:
  # Priority is an integer defining the relative importance of this Localization compared to others.
  # Lower numbers are considered lower priority.
  # Override values in lower Localization will be overridden by those in higher Localization.
  # (Optional) Default priority is 500.
  priority: 300
  feed:
    apiVersion: apps/v1
    kind: Deployment
    name: my-nginx
    namespace: foo
  overrides: # defines all the overrides to be processed with
    - name: add-update-labels
      type: MergePatch
      # Value is a YAML/JSON format patch that provides MergePatch to current resource defined by feed.
      # This override adds or updates some labels.
      value: '{"metadata":{"labels":{"deployed-in-cluster":"clusternet-5l82l"}}}'
    - name: scale-replicas
      type: JSONPatch
      # Value is a YAML/JSON format patch that provides JSONPatch to current resource defined by feed.
      # This patch sets replicas to 1.
      # But due to lower priority, this value will be overridden by above "nginx-local-overrides-demo-higher-priority" eventually.
      value: |-
        [
          {
            "path": "/spec/replicas",
            "value": 1,
            "op": "replace"
          }
        ]
```

更多示例

注册一个集群

- <https://github.com/clusternet/clusternet/blob/main/docs/tutorials/installing-clusternet-with-helm.md>

部署一个完整的多集群应用

- <https://github.com/clusternet/clusternet/blob/main/docs/tutorials/deploying-applications-to-multiple-clusters.md>

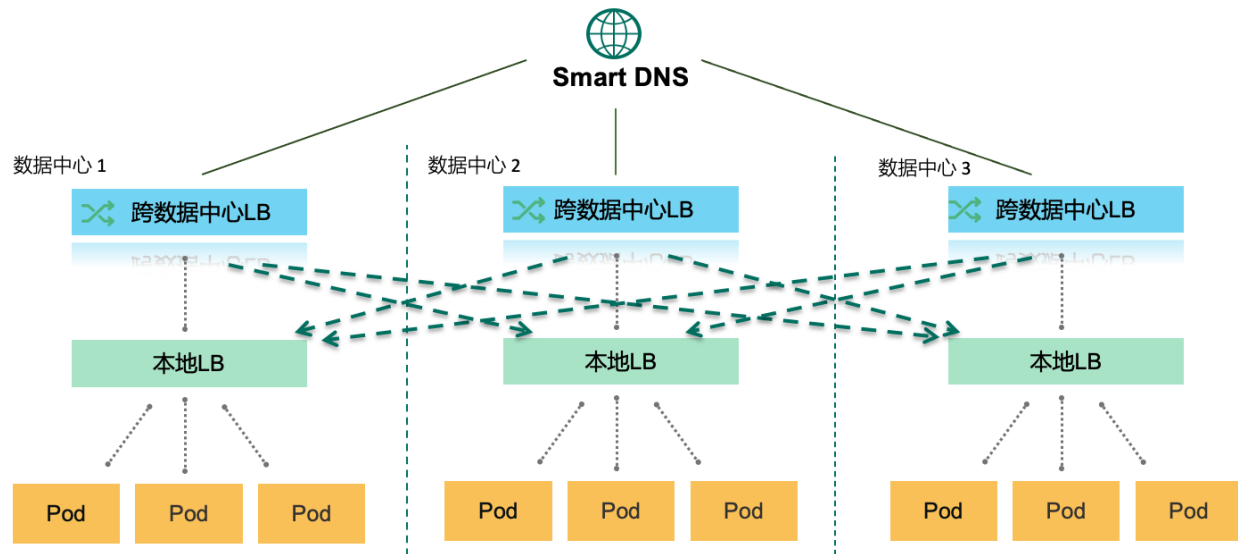
如何使用 client-go 创建多集群应用

- <https://github.com/clusternet/clusternet/blob/main/examples/clientgo/README.md>

3. Istio 多集群

跨地域流量管理的挑战

- 采用多活数据中心的网络拓扑，任何生产应用都需要完成跨三个数据中心的部署。
- 为满足单集群的高可用，针对每个数据中心，任何应用都需进行多副本部署，并配置负载均衡。
- 以实现全站微服务化，但为保证高可用，服务之间的调用仍以南北流量为主。
- 针对核心应用，除集群本地负载均衡配置以外，还需配置跨数据中心负载均衡，并通过权重控制将 99% 的请求转入本地数据中心，将 1% 的流量转向跨地域的数据中心。



规模化带来的挑战

3 主数据中心, 20 边缘数据中心, 100+ Kubernetes 集群

规模化运营 Kubernetes 集群

- 总计 100,000 物理节点
- 单集群物理机节点规模高达 5,000

业务服务全面容器化, 单集群

- Pod 实例可达 100,000
- 发布服务 5,000-10000

单集群多环境支持

- 功能测试、集成测试、压力测试共用单集群
- 不同环境需要彼此隔离

异构应用

- 云业务, 大数据, 搜索服务
- 多种应用协议
- 灰度发布

日益增长的安全需求

- 全链路 TLS

可见性需求

- 访问日志
- Tracing

多集群部署

Kubernetes 集群联邦

- 集群联邦 APIServer 作为用户访问 Kubernetes 集群入口
- 所有 Kubernetes 集群注册至集群联邦

可用区

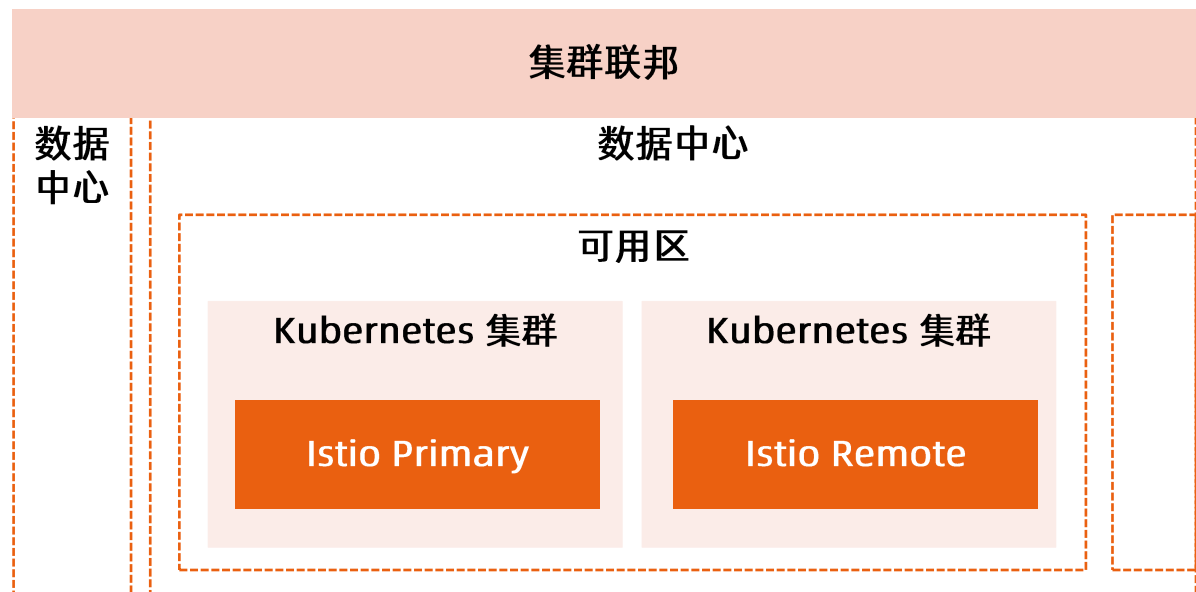
- 数据中心中具有独立供电制冷设备的故障域
- 同一可用区有较小网络延迟
- 同一可用区部署了多个 Kubernetes 集群

多集群部署

- 同一可用区设定一个网关集群
- 网关集群中部署 Istio Primary
- 同一可用区的其他集群中部署 Istio Remote
- 所有集群采用相同 RootCA
- 相同环境 TrustDomain 相同

东西南北流量统一管控

- 同一可用区的服务调用基于 Sidecar
- 跨可用区的服务调用基于 Istio Gateway

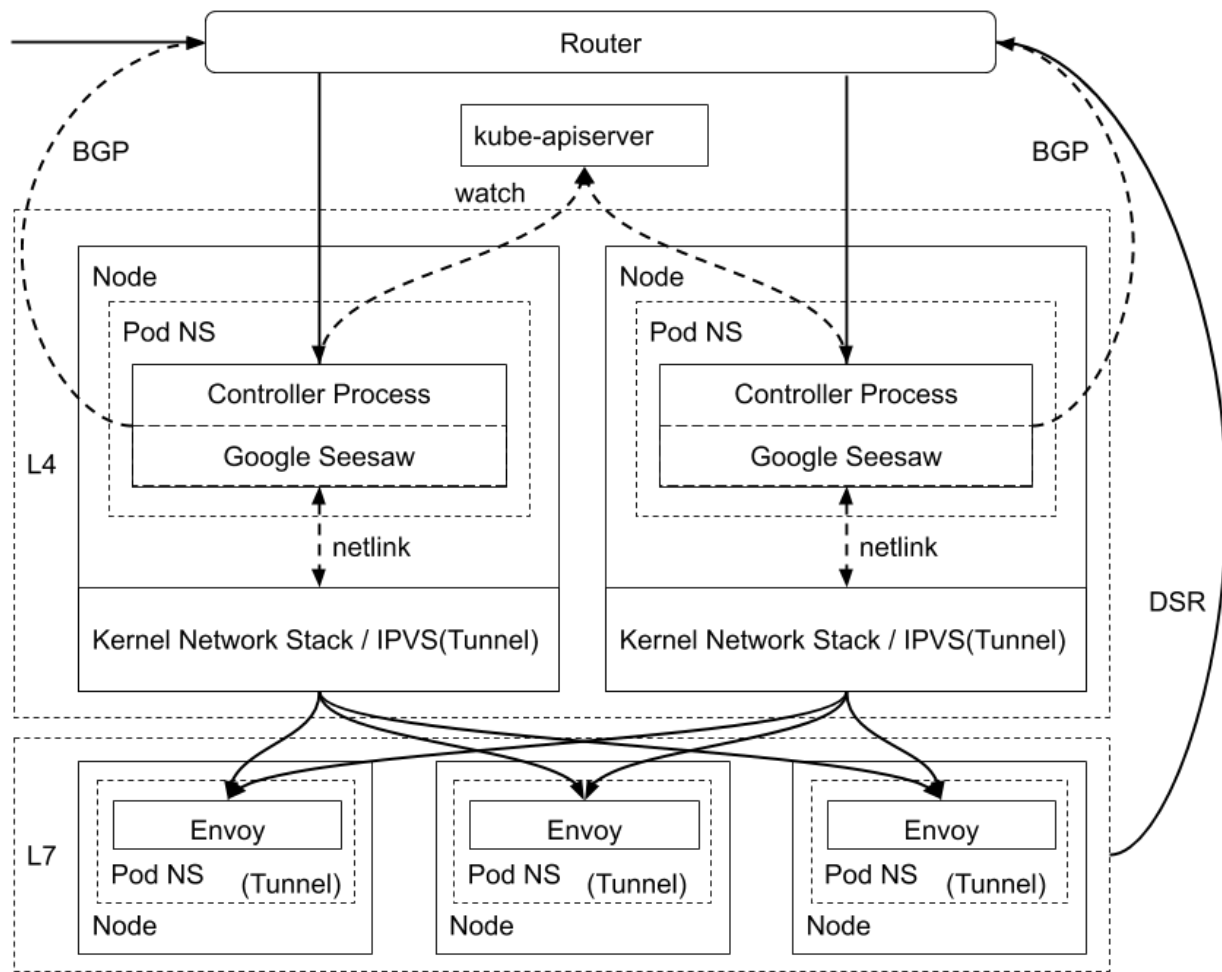


入站流量架构 L4 + L7

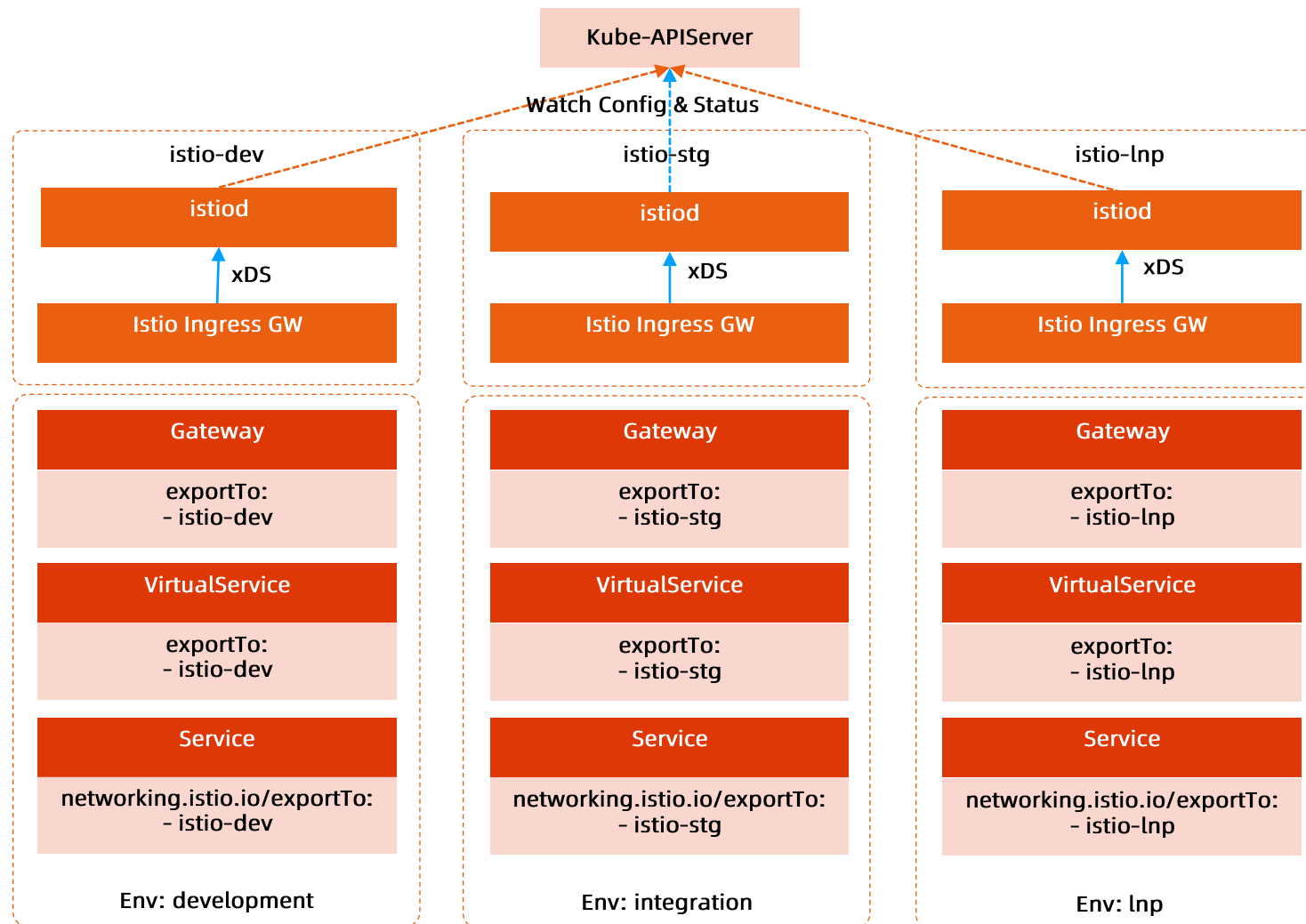
为不同应用配置独立的网关服务以方便网络隔离。

基于 IPVS/xDP 的 Service Controller:

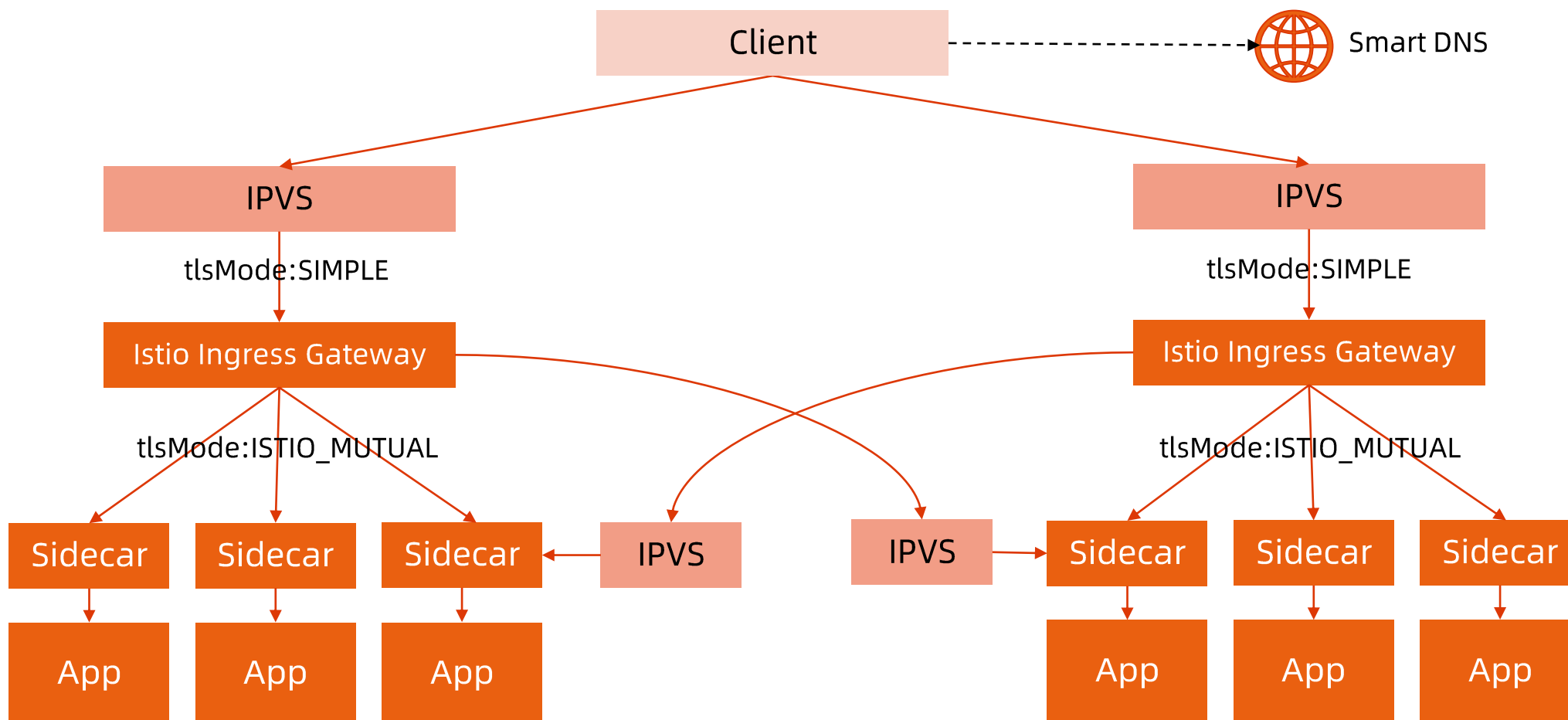
- 四层网关调度;
- 虚拟 IP 地址分配;
- 基于 IPIP 协议的转发规则配置;
- 基于 BGP 的 IP 路由宣告;
- 在 Ingress Pod 中配置 Tunnel 设备, 并绑定虚拟 IP 地址以卸载 IPIP 包。



单网关集群多环境支持



应用高可用接入方案



为应用发布服务

定义 `LoadBalancer Type Service`，提供集群外可访问的 `LoadBalancerIP`。

其他集群可通过定义 `WorkloadEntry` 指向该 `LoadBalancerIP`，以实现故障转移目的。

创建 WorkloadEntry

创建 WorkloadEntry 指向其他数
据中心 LoadBalancerIP

```
apiVersion: networking.istio.io/v1beta1
kind: WorkloadEntry
metadata:
  name: foo
spec:
  address: foo.bar.svc.cluster2
labels:
  run: foo
locality: region1/zone1
```


定义 ServiceEntry

同时选择 WorkloadEntry 和本地 Pod。

ServiceEntry 对象可以将本地 Pod 和具有相同 Label 的 WorkloadEntry 定义成相同的 Envoy Cluster。

```
apiVersion: networking.istio.io/v1beta1
kind: ServiceEntry
metadata:
  name: foo
spec:
  hosts:
  - foo.com
  ports:
  - name: http-default
    number: 80
    protocol: HTTP
    targetPort: 80
  resolution: STATIC
  workloadSelector:
    labels:
      run: foo
```

在 VirtualService 中引用 ServiceEntry

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: foo
spec:
  gateways:
  - foo
  hosts:
  - foo.com
  http:
  - match:
    - port: 80
    route:
    - destination:
        host: foo.com
        port:
          number: 80
```

```
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: foo
spec:
  selector:
    istio: ingressgateway
  servers:
  - hosts:
    - foo.com
    port:
      name: http-default
      number: 80
      protocol: HTTP
```

为 workload 添加 Locality 信息

Istio 从如下配置中读取，基于这些配置，我们可以为 Istio 中运行的所有 workload 添加地域属性。

- **Kubernetes Node 对象中的地域信息，所有 Pod 自动继承该 Locality 信息**
 - region: topology.kubernetes.io/region
 - zone: topology.kubernetes.io/zone
 - subzone: topology.istio.io/subzone
- **Kubernetes Pod 的 istio-locality 标签，可覆盖节点 Locality 信息**
 - istio-locality: "region/zone/subzone "
- **WorkloadEntry 的 Locality 属性**
 - locality: region/zone/subzone

定义基于 Locality 的流量转发规则

Distribute

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: foo
spec:
  host: foo.com
  trafficPolicy:
    loadBalancer:
      localityLbSetting:
        distribute:
          - from: "*"/*"
            to:
              region1/zone1/*: 99
              region2/zone2/*: 1
        enabled: true
    outlierDetection:
      baseEjectionTime: 10s
      consecutive5xxErrors: 100
      interval: 10s
  tls:
    mode: ISTIO_MUTUAL
```

Failover

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: foo
spec:
  host: foo.com
  trafficPolicy:
    loadBalancer:
      localityLbSetting:
        enabled: true
        failover:
          - from: region1/zone1
            to: region2/zone2
    outlierDetection:
      baseEjectionTime: 10m
      consecutive5xxErrors: 1
      interval: 2s
  tls:
    mode: ISTIO_MUTUAL
```

应对规模化集群挑战

Istio xDS 默认发现集群中所有的配置和服务状态，在超大规模集群中，Istiod 或者 Envoy 都承受比较大的压力。

- 集群中的有 10000Service，每个 Service 开放 80 和 443 两个端口，Istio 的 CDS 会 discover 出 20000 个 Envoy Cluster。
- 如果开启多集群，Istio 还会为每个 cluster 创建符合域名规范的集群。
- Istio 还需要发现 remote cluster 中的 Service，Endpoint 和 Pod 信息，而这些信息的频繁变更，会导致网络带宽占用和控制面板的压力都很大。

meshConfig 中控制可见性：

```
defaultServiceExportTo:  
- "."  
defaultVirtualServiceExportTo:  
- "."  
defaultDestinationRuleExportTo:  
- "."
```

通过 Istio 对象中的 exportTo 属性覆盖默认配置。

Istiod 自身的规模控制

社区新增加了 discoverySelector 的支持，允许 Istiod 只发现添加了特定 label 的 namespaces 下的 Istio 以及 Kubernetes 对象。

但因为 Kubernetes 框架的限制，改功能依然要让 Istiod 接收所有配置和状态变更新细，并且在 Istiod 中进行对象过滤。在超大集群规模中，并未降低网络带宽占用和 Istiod 的处理压力。

需要继续寻求从 Kubernetes Server 端过滤的解决方案。

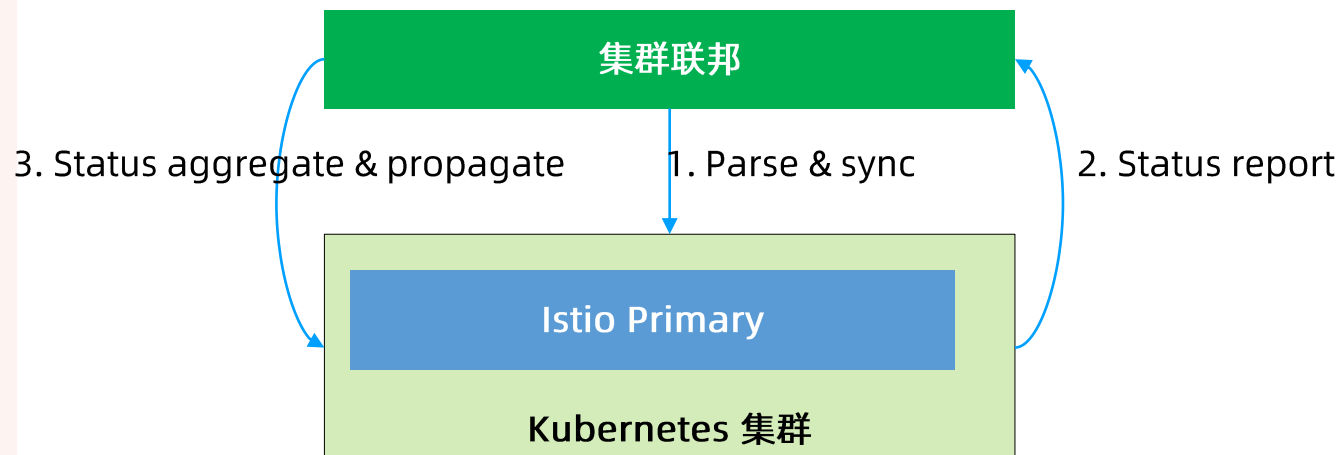
基于联邦的统一流量模型

Spec

- Scope
- TrafficTemplate
 - Istio models
 - Kubernetes Services
- Override
 - 可为不同目标集群修改模板属性值
 - 支持多种 override 方法
 - jsonPatch
 - mergePatch
- Policy
 - PlacementPolicy
 - RolloutPolicy
 - RuntimePolicy

Status

- Conditions
 - 四层网关状态
 - 七层网关配置完成度
 - 证书版本
 - 网关服务 IP 和 FQDN



统一流量模型 - NameService

Spec

- Global Name FQDN
- TTL
- DNSPolicy
 - RoundRobin
 - Locality
 - Ratio
- HeathCheck Port
- Target
 - Target Service FQDN
 - Ratio

Status

- Conditions
 - 域名配置结果
- 配置错误信息

AccessPoint 控制器

- **PlacementPolicy** 控制，用户可以选择目标集群来完成流量配置，甚至可以选择关联的 **FederatedDeployment** 对象，使得 **AccessPoint** 自动发现目标集群并完成配置。
- 完成了状态上报，包括网关虚拟 **IP** 地址，网关 **FQDN**，证书安装状态以及版本信息，路由策略是否配置完成等。这补齐了 **Istio** 自身的短板，使得任何部署在 **Istio** 的应用的网络配置状态一目了然。
- 发布策略控制，针对多集群的配置，可实现单集群的灰度发布，并且能够自动暂停发布，管理员验证单个集群的变更正确以后，再继续发布。通过此机制，避免因为全局流量变更产生的故障。
- 不同域名的 **AccessPoint** 可拥有不同的四层网关虚拟 **IP** 地址，以实现基于 **IP** 地址的四层网络隔离。
- 控制器可以基于 **AccessPoint** 自动创建 **WorkloadEntry**，并设置 **Locality** 信息。

未来展望

- 全面构建基于 Mesh 的流量管理
- 在用户无感知的前提下将南北流量转成东西流量
- 数据平面加速 Cilium

THANKS

 极客时间 | 训练营